

A
Project Report
On

BLOCKCHAIN-BASED CROWDFUNDING APPLICATION

*Submitted in partial fulfillment of the requirements for the award
of the degree of*

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Session: 2022 - 2026

Submitted by:

KANHAIYA AGARWAL	(22013440032)
KHUSHI SINGH	(22013440033)
KUMARI KHUSHI SINGH	(22013440034)
KUNAL KUMAR	(22013440035)

Under the esteemed guidance of

Prof. Jeevan Kumar
Dept. of Computer Science and Engineering



Department Of Computer Science and Engineering
RVS COLLEGE OF ENGINEERING AND TECHNOLOGY
JAMSHEDPUR, JHARKHAND – 831012



**Approved by AICTE, New Delhi and Affiliated to Jharkhand University of
Technology, Ranchi**

CANDIDATES' DECLARATION

We declare that this written submission represents our ideas in our own words and where others ideas or words have been included. We have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Kanhaiya Agarwal
(22013440032)

Khushi Singh
(22013440033)

Kumari Kumari Singh
(22013440034)

Kunal Kumar
(22013440035)

This is to certify that the above statement made by the candidate is true and correct to the best of my/our knowledge and belief.

Prof. Jeevan Kumar
Department of CSE
RVSCET, Jamshedpur
Project Guide

Date: / / 2026

RVS COLLEGE OF ENGINEERING AND TECHNOLOGY
JAMSHEDPUR, JHARKHAND – 831012



CERTIFICATE

This is to certify that the project report entitled “Blockchain-Based Crowdfunding Application” submitted by Kanhaiya Agarwal (Regd. No.:- 22013440032), Khushi Singh (Regd. No.:- 22013440033), Kumari Khushi Singh (Regd. No.:- 22013440034), Kunal Kumar (Regd. No.:- 22013440035), for the partial fulfillment of the requirement for the award of “BACHELOR OF TECHNOLOGY” in COMPUTER SCIENCE AND ENGINEERING DEPARTMENT (Session: 2022-2026) has been carried out under my supervision and guidance. It is certified that the report embodies the results of the work carried out by them within the prescribed period.

Prof. Jeevan Kumar
Department of CSE
RVSCET, JSR
Project Guide

Prof. Jeevan Kumar
H.O.D, CSE

Signature of External

Date: / / 2026

Place: RVSCET, Jamshedpur

ACKNOWLEDGEMENT

We would like to take this opportunity to express our sincere gratitude to **Prof. Jeevan Kumar** for his continuous guidance, valuable supervision, and consistent encouragement throughout the course of this project. His insightful suggestions and constructive feedback played a crucial role in the successful completion of this work.

We also extend our heartfelt thanks to all the faculty members and supporting staff of the **Department of Computer Science and Engineering** for their cooperation, timely assistance, and valuable suggestions, which greatly contributed to the smooth progress of the project.

Kanhaiya Agarwal
(22013440032)

Khushi Singh
(22013440033)

Kumari Khushi Singh
(22013440034)

Kunal Kumar
(22013440035)

ABSTRACT

Crowdfunding has emerged as an effective mechanism for raising capital; however, traditional crowdfunding platforms suffer from limitations such as lack of transparency, centralized control, high intermediary fees, and trust-related issues. To address these challenges, this project presents a **Blockchain-based Crowdfunding Platform** that leverages decentralized technologies to ensure transparency, security, and trust among participants. The system is developed using Web3 technologies, where smart contracts automate fund collection, management, and distribution without the involvement of intermediaries. Contributors can securely connect their digital wallets to fund campaigns, while project creators can launch campaigns with predefined conditions embedded within smart contracts. Blockchain ensures immutability of transactions, enabling contributors to track fund usage in real time and reducing the risk of fraud or misuse. Additionally, the platform supports secure authentication, role-based access control, and seamless interaction between the frontend and blockchain network. By eliminating centralized authorities and enabling trustless transactions, the proposed system enhances accountability, minimizes operational costs, and promotes fair fundraising practices. This project demonstrates how decentralized applications (DApps) can revolutionize crowdfunding by providing a transparent, reliable, and efficient alternative to conventional platforms.

Keywords: *Blockchain, Smart Contracts, Crowdfunding, Security, Ethereum*

TABLE OF CONTENTS

CANDIDATES' DECLARATION	I
ACKNOWLEDGEMENT	III
ABSTRACT	IV
TABLE OF CONTENTS	V
LIST OF FIGURES	VI
LIST OF TABLES	VII
Chapter 1: INTRODUCTION	(1 – 12)
1.1 MOTIVATION	1 - 3
1.2 LITERATURE SURVEY	3 - 7
1.3 PROBLEM STATEMENT	7 – 9
1.4 AIM AND OBJECTIVE OF PROJECT	10 - 12
Chapter 2: ESSENTIAL PRELIMINARIES	(13 – 21)
2.1 PRELIMINARIES OF THEORY	13 - 18
2.2 PRELIMINARIES IN MATHEMATICS	18 – 20
2.3 SYSTEM BLOCK DIAGRAM AND CORE INFRASTRUCTURE	21
Chapter 3: SYSTEM DESIGN AND SPECIFICATIONS	(22 – 34)
3.1 HIGH LEVEL DESIGN (HLD)	22 - 32
3.2 LOW LEVEL DESIGN (LLD)	32 - 34
Chapter 4: TESTING	(35 – 38)
4.1 UNIT TESTING	35 - 37
4.2 INTEGRATION TESTING	37 - 38
Chapter 5: CONCLUSION AND LIMITATIONS	39
5.1 CONCLUSION	39
5.2 LIMITATIONS	39
APPENDIX A - SOURCE CODE	40 - 45
APPENDIX B - SCREEN SHOTS	46 - 48
REFERENCES	49

LIST OF FIGURES

Figure 2.1: System Block Diagram of the Blockchain-Based Crowdfunding Platform.....	21
Figure 3.1: Project Model	23
Figure 3.2: Waterfall Model	24
Figure 3.3: Structure Chart	26
Figure 3.4: Level 0 DFD	27
Figure 3.5: Level 1 DFD	28
Figure 3.6: E-R Diagram	30
Figure 3.7: UML (Use Case Diagram)	32
Figure B.1: Home Page	46
Figure B.2: MetaMask Wallet	46
Figure B.3: Create Campaign Page	47
Figure B.4: Campaign Details Page	47
Figure B.5: Withdraw / Refund Page	48
Figure B.6: Payment Transaction Page	48

LIST OF TABLES

Table 2.1: Key Properties of Blockchain Technology vs. Centralized Database Systems	13
Table 2.2: Ethereum Gas Cost Comparison across Smart Contract Functions.....	20
Table 3.1: Campaign Entity Attributes and On-Chain Data Type Mappings	30
Table 3.2: Smart Contract Module Functions, Visibility, and Access Control Summary	33
Table 4.1: Unit test case for Smart contract core functions and expected outcomes.....	36 – 37
Table 4.2: Integration Test Scenarios for Frontend-to-Blockchain Workflow Validation	38

INTRODUCTION

1.1 MOTIVATION

Fundraising has always been at the heart of turning ideas into reality. Whether it is a student with a startup dream, a social entrepreneur trying to uplift a community, a researcher seeking support for ground-breaking work, or a small business owner looking for early backing — the need for accessible, trustworthy, and efficient funding is universal. The ability to raise capital without being limited by geography, social connections, or institutional gatekeepers has the potential to democratize opportunity in a way that was simply not possible a generation ago. Crowdfunding emerged as one of the most promising answers to this need, giving ordinary people the ability to back causes and projects they believe in, directly and without friction.

But before even discussing the technology, there is a more fundamental question worth addressing — one that often goes unasked: why would anyone voluntarily give their money to a stranger's project? What motivates a person to open their wallet for a campaign they did not ask to see, created by someone they have never met, for a goal that may or may not succeed?

The answer lies in something deeply human. People donate, back, and fund campaigns because they see a piece of themselves in the cause. A student who once struggled to afford equipment backs a campaign for affordable learning tools. Someone who lost a loved one to a rare disease contributes to a medical research fundraiser. An early believer in a product idea becomes a backer because they want to see that idea exist in the world — and they want to be part of making it happen. Crowdfunding, at its best, is not charity. It is collective ownership of an idea. It is the feeling of being on the ground floor of something meaningful, of saying "this matters and I helped make it real."

Beyond emotion, there is also rational motivation. Many crowdfunding campaigns offer rewards, early access, equity, or recognition in return for contributions. Backers are not always donating out of pure goodwill — they are often making a calculated bet that the project will deliver value, and they want in early. This mix of emotional and rational

motivation is precisely what makes crowdfunding such a powerful model for raising capital outside of traditional financial institutions.

Yet the platforms that exist today to facilitate this process are far from perfect. Traditional crowdfunding websites are built on centralized architectures that place enormous power in the hands of a single company. Fees eat into the funds raised, campaigns can be removed without notice, payouts can be delayed or denied based on opaque internal policies, and contributors have no real way of knowing whether their money is being used as promised. The platform sits in the middle of every transaction, acting as an intermediary that both parties are forced to trust — not because trust has been earned, but because there is simply no alternative.

And this is where the model begins to break down. Because the very thing that motivates someone to donate — trust in the cause, belief that their money will be used as intended — is precisely what centralized platforms cannot guarantee. Stories of people contributing to campaigns only to later find out that funds were misused, siphoned off, or withheld due to vague policy violations are not hard to come by. Creators lose months of effort when campaigns are taken down without clear explanation. Contributors lose money with little to no recourse. There is a fundamental accountability gap at the core of how these platforms operate, and it has persisted for years because the underlying architecture makes transparency structurally difficult. When a single centralized entity controls the ledger, the rules, and the payout process, abuse becomes a matter of opportunity rather than exception.

At the same time, blockchain technology and what it promises — a world where trust does not depend on a central authority, but is instead built into the system itself — presents a genuinely compelling alternative. The emergence of smart contracts made it possible to encode the rules of an agreement directly into executable code that runs automatically and cannot be tampered with after deployment. No third party needs to verify, approve, or oversee the transaction. The logic is transparent, the execution is deterministic, and the outcome is recorded permanently on a public ledger that anyone in the world can audit at any time.

The connection between these two worlds — the broken promises of centralized crowdfunding and the trustless guarantees of blockchain — is what motivated this project. Why should a backer have to trust a platform or even a campaign creator blindly,

when the rules of the campaign can be written into a smart contract that executes exactly as defined? Why should funds be sitting in a company's bank account, subject to their discretion, when they can live in a contract that only releases them under conditions the contributors themselves have agreed to and voted on?

These questions shaped the core design of this project. The goal was to build a crowdfunding application where the entire fund flow — from the moment a contribution is made to the moment a withdrawal is processed — is governed by smart contracts deployed on the Ethereum blockchain. There is no intermediary collecting fees, no platform that can freeze a campaign arbitrarily, and no single point of failure or manipulation. Every transaction is recorded on an immutable ledger that any contributor can verify independently. Creators can launch campaigns with predefined funding goals and timelines, contributors can back campaigns using their digital wallets, and any spending request made by a campaign creator must pass through a community voting process before funds are released.

Beyond the technical challenge, there is a broader relevance to this project that extends well past the academic context. The post-pandemic world accelerated a shift toward remote collaboration, digital-first interactions, and borderless financial activity. People increasingly expect to participate in economic activity without being constrained by national borders, banking infrastructure, or institutional gatekeeping. A blockchain-based crowdfunding platform is not just a technical experiment — it is a response to a real and growing demand for financial tools that are open, verifiable, and resistant to centralized abuse.

Web3 tools are mature enough today to support real, production-grade applications, yet most people still interact with Web2-era platforms that have not fundamentally changed in over a decade. There is a clear and widening gap between what the technology is capable of and what users are actually offered. Decentralization should not remain a concept confined to whitepapers and developer forums. It should show up in working software that solves real problems for real people. This project is an attempt to make that case — not just in theory, but in practice. That belief shaped every design decision, every architectural choice, and every line of code written over the course of this work.

1.2 LITERATURE SURVEY

The literature review began with an exploration of how fundraising mechanisms and financial trust have evolved in the digital era, and how decentralized technologies have emerged as a credible response to the structural failures of centralized crowdfunding platforms. Traditional crowdfunding methods — involving intermediary-controlled fund flows, opaque payout policies, and limited contributor accountability — have been studied extensively, and the evidence consistently points toward a need for trustless, transparent alternatives.

➤ **Traditional Crowdfunding and Its Limitations:**

[1] Mollick (2014) conducted one of the earliest large-scale empirical studies on crowdfunding dynamics, drawing on a dataset of over 48,500 Kick-starter projects with combined funding exceeding \$237 million. Published in the *Journal of Business Venturing* (Vol. 29, pp. 1–16), the study found that personal networks and underlying project quality are the strongest predictors of success, but also highlighted that over 75% of successfully funded campaigns delivered their products later than promised. This finding points to a fundamental accountability gap — once funds are raised on centralized platforms, contributors have no enforceable mechanism to ensure timely or appropriate use of their money.

[2] Agrawal, Catalini, and Goldfarb (2015) examined the geographical and social dimensions of crowdfunding in their paper "Crowdfunding: Geography, Social Networks, and the Timing of Investment Decisions," published in the *Journal of Economics & Management Strategy* (Vol. 24, No. 2, pp. 253–274). Their research found that although online platforms reduce many distance-related barriers, local and distant funders exhibit fundamentally different funding patterns. Local funders, who often have pre-existing offline relationships with campaign creators, tend to contribute earlier and are less dependent on platform-level signals. This research highlights how trust remains a social and relational construct — one that centralized platforms are poorly equipped to formalize or enforce at scale.

➤ **Blockchain Technology — Architecture and Potential:**

[3] Zheng, Xie, Dai, Chen, and Wang (2018) published a comprehensive survey titled "Blockchain Challenges and Opportunities: A Survey" in the *International Journal of Web and Grid Services* (Vol. 14, No. 4, pp. 352–375). The paper provides a detailed taxonomy of blockchain architecture, consensus mechanisms, and application domains, and identifies smart contracts as one of the most transformative components of blockchain infrastructure. The authors note that blockchain's core properties — immutability, decentralization, and transparency — make it particularly well-suited for applications where trust between unknown parties is required. This directly supports the design rationale of using Ethereum and Solidity-based smart contracts to govern fund flows in this project.

[4] Casino, Dasaklis, and Patsakis (2019) conducted a systematic literature review of blockchain-based applications across multiple domains, published in *Telematics and Informatics* (Vol. 36, pp. 55–81). Analysing 260 research papers published between 2014 and 2018, their review classified blockchain applications across sectors including finance, supply chain, healthcare, and governance. Their findings established that decentralization is not merely a technical feature but a structural shift in how accountability is enforced — replacing the need for trusted intermediaries with cryptographic verification. This framework directly informed the decision to eliminate platform-level intermediaries in the fund management architecture of this project.

➤ **Smart Contracts and Solidity Development:**

[5] Nakamoto (2008) introduced the foundational concept of a trustless peer-to-peer electronic cash system using a distributed ledger in the Bitcoin whitepaper, establishing the principle that financial agreements can be enforced through cryptographic proof rather than centralized authority. Building on this, Buterin (2014) proposed Ethereum as a programmable blockchain capable of executing arbitrary smart contract logic, enabling developers to encode the full terms of an agreement — including conditions for fund release — directly into deployed code that cannot be altered post-deployment. These two foundational works collectively establish the theoretical basis for the entire smart contract infrastructure used in this project.

[6] A research paper published at the *2020 IEEE International Conference on Advances in Computing and Communication Engineering* proposed a smart contract-based crowdfunding platform using Solidity on Ethereum, demonstrating that contributor-controlled fund release mechanisms can be implemented effectively on-chain. The study validated that milestone-based and voting-based withdrawal systems reduce the risk of fund misuse compared to discretionary creator-controlled models, a finding that directly shaped the voting mechanism implemented in this project.

➤ **Development Tooling — Hardhat And Testing Frameworks:**

[7] Research on smart contract mutation testing by Wu et al. (2019), published on arXiv (arXiv:1908.03707), examined systematic testing approaches for Ethereum smart contracts and demonstrated that structured testing environments — where contract behaviour is validated against defined mutation operators — are essential for catching logic errors before deployment. The study showed that untested or minimally tested contracts are significantly more vulnerable to unexpected behaviour. Hardhat, the development environment used in this project, provides precisely this kind of structured local testing infrastructure, with built-in support for Mocha and Chai assertion libraries, Solidity stack traces, and gas consumption reporting — enabling rigorous pre-deployment validation of all contract functions.

➤ **Wallet-Based Authentication — Metamask:**

[8] Frunzete and Stoichescu (2023) published "A Secure and Decentralized Authentication Mechanism Based on Web 3.0 and Ethereum Blockchain Technology" in *Applied Sciences* (Vol. 13, No. 4, Article 2231, MDPI). The paper proposes and validates a Web3 authentication model in which MetaMask serves as the primary interface between users and smart contracts deployed on Ethereum. The authors demonstrate that wallet-based authentication — where users prove ownership of an account by signing cryptographic messages with their private key — provide stronger security guarantees than traditional session-based or OAuth-based authentication, as no credentials are stored on any server. This approach was adopted in this project to handle contributor and campaign creator authentication without any centralized identity management.

➤ **Frontend Integration — React.Js and Thirdweb SDK:**

[9] Research and developer documentation on the Thirdweb SDK (2022–2023) has demonstrated the value of abstraction layers between deployed smart contracts and React.js-based frontends. Rather than requiring raw Ethers.js or Web3.js calls for every contract interaction, the SDK provides hook-based interfaces that integrate natively with React's component and state management lifecycle. This significantly reduces frontend complexity and the likelihood of integration-level bugs, particularly around wallet connection handling, transaction state management, and event listening — all of which are central to the contributor and campaign creator workflows in this project.

➤ **Prior Decentralized Crowdfunding Research and Identified Gaps:**

[10] A study published in the *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* (November 2022) proposed a blockchain-based crowdfunding DApp on Ethereum that stores campaign details, contributions, and withdrawal records on-chain using a shared ledger. While the platform demonstrated the feasibility of eliminating intermediaries and improving transparency, the study acknowledged that most existing implementations lack a contributor-voting mechanism for spending approval, leaving campaign creators with unilateral control over raised funds after the goal is met.

Based on the literature reviewed, the following gaps were identified in prior work:

- a) Most research proposes theoretical models or partial contract implementations without a complete, deployable frontend connected to the blockchain.
- b) Existing prototypes assume technical familiarity with blockchain, creating accessibility barriers for non-technical contributors.
- c) The majority of systems reviewed lack an on-chain voting mechanism, allowing campaign creators to withdraw funds freely once a goal is reached.
- d) Limited use of current SDK tooling such as Thirdweb means that frontend-to-contract integration in earlier work is unnecessarily complex and harder to maintain.

e) Few studies validate contract behaviour through full testnet deployment, leaving deployment-specific issues — such as gas estimation errors and network-specific transaction handling — unaddressed.

This project was designed to address each of these gaps through an end-to-end implementation using React.js, Thirdweb SDK, Solidity, Hardhat, MetaMask, and the Sepolia Testnet.

1.3 PROBLEM STATEMENT

Crowdfunding has emerged as one of the most accessible and democratic methods of raising capital in the modern era. The ability to gather small contributions from a large number of people over the internet has opened doors for entrepreneurs, artists, social causes, and innovative projects that would otherwise struggle to secure funding through traditional financial institutions. However, despite the growing adoption of crowdfunding as a fundraising model, the platforms that facilitate this process are built on centralized architectures that introduce a wide range of structural, financial, and ethical problems — problems that have persisted for years without any fundamental solution.

In the conventional crowdfunding model, a campaign creator registers on a platform, sets a funding goal and deadline, and collects contributions from backers who trust that the funds will be used as described. The platform acts as the intermediary — holding the funds, processing transactions, enforcing its own policies, and deciding when and how payouts are made. Each of these stages involves the platform exercising significant control over both the creator and the contributor. For projects that successfully meet their goals, the platform collects a percentage of the total amount raised as fees, often ranging from 5% to 10% of the total funds. For projects that fail or get suspended, contributors may lose their money with little or no recourse.

One of the primary issues with the current system is the complete absence of financial transparency. Once a contributor transfers funds to a campaign on a centralized platform, they have no way to track how those funds are being held, managed, or spent. The platform operates as a black box — contributors are expected to trust the creator blindly, and creators are expected to trust the platform blindly. There is no mechanism for contributors to verify fund movements, no on-chain record of transactions, and no enforceable condition that ensures the money is used for its stated purpose. Cases of fund

misuse, fraudulent campaigns, and sudden platform-initiated freezes on legitimate campaigns are widely documented and continue to damage contributor confidence globally.

Another significant problem is the concentration of power in a single centralized entity. Platforms such as Kickstarter, Indiegogo, and GoFundMe retain the right to remove campaigns, withhold payouts, and change their terms of service unilaterally. Campaign creators who have invested months of effort in building a community and running a campaign can have their work removed overnight without meaningful explanation or appeal. Contributors who have backed such campaigns lose their money without any transparent audit trail to understand what happened. This power imbalance is not a minor inconvenience — it is a structural flaw that makes centralized crowdfunding inherently unsuitable for use cases that require guaranteed accountability.

From the contributor's perspective, the current system offers almost no protection or participation beyond the act of donating. Once a contribution is made, the contributor has no formal voice in how the funds are used. There is no mechanism for contributors to approve or reject spending decisions made by the campaign creator. If a creator decides to redirect funds toward purposes not originally stated in the campaign, contributors have no on-chain or contractual basis to challenge it. The entire post-contribution experience is characterized by a lack of control, lack of visibility, and an over-reliance on the goodwill of both the platform and the creator.

In recent years, several blockchain-based fundraising experiments have emerged, but many of them suffer from significant limitations. Some focus solely on token-based fundraising without addressing fund governance. Others implement basic smart contracts that accept contributions but do not include any mechanism for controlled withdrawal or contributor oversight. There is also a significant gap in solutions that offer an end-to-end decentralized experience — from campaign creation and contribution to spending request submission and community-based approval — all within a single, deployable, and user-friendly application. Furthermore, most existing prototypes are not connected to functional frontends, making them inaccessible to non-technical users.

The absence of a unified, transparent, and decentralized crowdfunding platform that eliminates intermediaries, enforces contributor rights through smart contracts, and provides an accessible interface for both campaign creators and backers presents a major

gap in the current Web3 and fundraising landscape. This project directly addresses that gap.

1.4 AIM AND OBJECTIVE OF PROJECT

The aim of the present project is to provide a decentralized, blockchain-powered crowdfunding platform that transforms the traditional fundraising process. The platform is envisioned as a transparent and trustless system that automates fund management and contributor oversight using Ethereum smart contracts, while offering an accessible and intuitive interface for both campaign creators and backers. The goal is to design a solution that is secure, transparent, and community-governed, effectively overcoming the structural drawbacks of centralized crowdfunding platforms, and contributing to a fairer, more accountable, and intermediary-free fundraising ecosystem.

Primary Objectives

1. Development of a smart contract-based fund management system

The core objective is to build and deploy a set of Solidity-based smart contracts on the Ethereum blockchain that govern the entire lifecycle of a crowdfunding campaign — from creation and contribution to spending request submission and fund withdrawal. The contracts will enforce campaign rules automatically, without any human intermediary. Contributors will be able to fund campaigns directly through their MetaMask wallets, and all transactions will be recorded immutably on the blockchain. The system will ensure that funds can only be withdrawn by campaign creators after a spending request has been submitted and approved by a majority of contributors through an on-chain voting mechanism, guaranteeing that no funds are misused or redirected without community consent.

2. Creation of a web-based decentralized application interface

To ensure ease of access and usability, the platform will feature a web-based frontend developed using React.js and integrated with the Thirdweb SDK for seamless blockchain interaction. The interface will support straightforward navigation for both campaign creators and contributors. Campaign creators will be able to launch new campaigns by specifying a funding goal, deadline, and campaign description. Contributors will be able to browse active campaigns, connect their MetaMask wallets, and make contributions

directly from the interface. The platform will provide real-time campaign status updates, contribution histories, and spending request details — making the entire fund management process visible and accessible to all participants without requiring any technical knowledge of blockchain.

3. Implementation of a contributor voting and withdrawal approval mechanism

A key component of the platform is the integration of a decentralized governance mechanism that gives contributors direct control over how raised funds are spent. After a campaign reaches its funding goal, the creator must submit a spending request describing the intended use of funds and the amount required. This request is then put to a vote among all contributors, where each contributor's voting weight is proportional to their contribution. A withdrawal can only be executed if a defined majority of contributors approve the request. This mechanism ensures that campaign creators remain accountable to their backers throughout the entire post-funding phase, and that no single party can unilaterally access the pooled funds.

4. Secure wallet-based authentication and transaction management

To ensure secure and trustless user interaction, the platform will implement MetaMask-based wallet authentication for all users. There will be no centralized login system, no stored passwords, and no server-side session management. Users will authenticate by connecting their Ethereum wallets and signing cryptographic messages, which proves ownership of their account without exposing any private credentials. All transactions — including contributions, spending request submissions, votes, and withdrawals — will be signed by the user's wallet and processed directly on the Ethereum blockchain, ensuring that every action is traceable, tamper-proof, and permanently recorded.

5. Deployment and testing on sepolia testnet

To validate the functionality and reliability of the smart contracts before any real-world usage, the entire platform will be deployed and tested on the Sepolia Testnet — an Ethereum proof-of-stake test network that closely simulates mainnet conditions. The Hardhat development framework will be used to compile, test, and deploy all contracts, with automated test suites covering all critical contract functions including campaign creation, contribution processing, spending request submission, voting logic, and fund

withdrawal. Gas consumption will be monitored and optimized during this phase to ensure that the platform remains cost-efficient for end users.

6. Elimination of intermediaries and platform fees

A central objective of this project is to build a crowdfunding experience that operates entirely without a central authority. Unlike existing platforms that charge fees of 5% to 10% on funds raised, the smart contract architecture of this platform ensures that all contributions flow directly from the contributor's wallet to the campaign contract, with no intermediary taking a cut. The platform has no ability to freeze campaigns, remove projects, or withhold funds — all rules are encoded in the contract and enforced automatically. This makes the platform equally accessible to anyone with an internet connection and an Ethereum wallet, regardless of geography, institutional affiliation, or financial background.

LONG-TERM VISION

In the long term, the platform is envisioned to evolve into a comprehensive decentralized fundraising ecosystem. Potential future developments include support for multiple blockchain networks beyond Ethereum, such as Polygon or BNB Chain, to reduce gas fees and improve transaction throughput for users in regions with limited financial infrastructure. The platform could also integrate token-based reward systems, where contributors receive campaign-specific tokens that represent their stake and entitle them to future benefits or equity. Decentralized identity verification mechanisms could be introduced to reduce the risk of fraudulent campaigns, while maintaining the pseudonymous nature of blockchain-based interaction.

Looking even further ahead, the platform has the potential to expand beyond project-based crowdfunding into broader decentralized finance use cases such as community treasury management, DAO-based grant distribution, and milestone-driven fund release for long-term initiatives. The inclusion of multilingual support, mobile wallet integration, and IPFS-based decentralized storage for campaign media and documentation would make the platform truly borderless and censorship-resistant. Ultimately, the goal is to empower individuals and communities to fund meaningful ideas without relying on any centralized authority — building a financial infrastructure that is open, fair, and owned by its users.

ESSENTIAL PRELIMINARIES

2.1 PRELIMINARIES OF THEORY

This chapter outlines the theoretical foundations that form the basis for the design and development of the decentralized crowdfunding platform. It explores the core concepts, technologies, and methodologies that were integral to the system's implementation. A fundamental understanding of blockchain technology, smart contracts, decentralized application architecture, and Web3 development tools is necessary to comprehend how the platform manages funds, enforces contributor rights, and operates without any central authority.

Table 2.1: Key Properties of Blockchain Technology vs. Centralized Database Systems

Property	Blockchain Technology	Centralized Database System
Data Control	Distributed — no single authority	Centralized — single entity in full control
Transparency	Fully transparent; all transactions public on ledger	Opaque; internal data hidden from users
Immutability	Immutable; records cannot be altered post-write	Mutable; records can be edited or deleted by admin
Trust Model	Trustless — enforced by cryptographic proof	Trust-based — relies on platform integrity
Intermediaries	None; peer-to-peer smart contract execution	Requires intermediary (platform, bank, etc.)
Transaction Fees	Gas fees per operation; network-dependent	Platform fees (5–10%) on total funds raised
Fault Tolerance	Highly resilient; no single point of failure	Single point of failure; downtime possible
Auditability	Open and permission less audit by any party	Restricted audit access; private ledgers
Scalability	Limited TPS (~15–30 on Ethereum mainnet)	High TPS; easily scaled with infrastructure
Deployment Cost	Higher upfront (contract deployment gas)	Lower upfront; managed by platform vendor

The system leverages Ethereum's programmable blockchain infrastructure, particularly Solidity-based smart contracts, to automate the rules governing campaign creation, fund collection, spending request submission, and withdrawal approval. The Thirdweb SDK is employed to bridge the gap between the React.js frontend and the deployed contracts, providing hook-based abstractions for wallet connection, contract interaction, and real-time transaction state management. MetaMask handles cryptographic identity and transaction signing, eliminating the need for any server-side authentication layer. Hardhat provides the local development and testing environment, ensuring that all contract logic is validated before deployment on the Sepolia Testnet.

2.1.1 BLOCKCHAIN TECHNOLOGY

Blockchain is a distributed ledger technology that stores data across a network of nodes in a series of cryptographically linked blocks. Each block contains a set of validated transactions, a timestamp, and a hash of the previous block — forming an immutable chain where no historical record can be altered without invalidating all subsequent blocks. This property makes blockchain fundamentally different from conventional databases, where a central administrator can modify or delete records at will.

In the context of this project, the Ethereum blockchain serves as the underlying infrastructure on which all platform data and logic resides. Every campaign creation, contributor transaction, spending request, vote, and withdrawal is recorded as an on-chain event that is permanently visible to all network participants. There is no server, no database administrator, and no company that controls this data. Once a transaction is confirmed on the Ethereum network, it becomes part of the permanent and publicly auditable record of the blockchain, providing a level of transparency and tamper-resistance that no centralized platform can match.

2.1.2 SMART CONTRACTS AND SOLIDITY

Smart contracts are self-executing programs stored on a blockchain that run automatically when predefined conditions are met. They are written in code, deployed to the network, and executed by every node in the network simultaneously — meaning their output is deterministic, verifiable, and not controlled by any single party. Once deployed, a smart contract cannot be modified, ensuring that the rules it encodes remain fixed and cannot be changed unilaterally by the platform operator or the campaign creator.

In this project, all smart contracts are written in Solidity — a statically typed, object-oriented programming language designed specifically for the Ethereum Virtual Machine (EVM). The primary contract governs the complete lifecycle of a crowdfunding campaign. It defines the logic for setting a funding goal and deadline, accepting contributions from multiple addresses, tracking each contributor's share, processing spending requests submitted by the campaign creator, conducting on-chain votes among contributors, and releasing funds only when a spending request has received majority approval. This means that the platform's core rules are not enforced by a company's terms of service — they are enforced by mathematics and code.

2.1.3 ETHEREUM AND THE ETHEREUM VIRTUAL MACHINE (EVM)

Ethereum is an open-source, programmable blockchain platform that extends the original Bitcoin model by enabling the deployment and execution of smart contracts. It introduced the concept of a world computer — a decentralized platform where arbitrary code can be executed by all nodes in the network simultaneously, producing identical results without requiring trust in any single executor. The Ethereum Virtual Machine (EVM) is the runtime environment in which all smart contracts on the Ethereum network are executed.

Ethereum uses Ether (ETH) as its native currency, which is also used to pay for computational operations on the network — commonly referred to as gas. Every function call on a smart contract, every transaction, and every state change on the Ethereum network consumes a certain amount of gas, which is paid by the user initiating the transaction. Gas consumption was carefully monitored during development using Hardhat's built-in gas reporting tools to ensure that all contract operations remain cost-efficient.

2.1.4 HARDHAT — DEVELOPMENT AND TESTING FRAMEWORK

Hardhat is a professional Ethereum development environment that provides a comprehensive suite of tools for compiling, testing, deploying, and debugging Solidity smart contracts. It includes a built-in local Ethereum network called the Hardhat Network, which allows developers to simulate blockchain behaviour on their own

machine without connecting to any live network or spending real Ether. This makes it possible to iterate rapidly during development, test all contract functions in isolation, and identify bugs before deployment.

In this project, Hardhat was used throughout the development lifecycle to compile all Solidity contracts and check for syntax and type errors, run automated test suites using Mocha and Chai that cover all critical contract functions including campaign creation, contribution logic, voting mechanisms, and withdrawal conditions, generate detailed Solidity stack traces when transactions revert, and produce gas consumption reports for each function. Deployment scripts written in JavaScript were also used to automate the process of deploying contracts to the Sepolia Testnet, ensuring consistent and reproducible deployments across environments.

2.1.5 THIRDWEB SDK

Thirdweb is a Web3 development platform that provides a suite of SDKs, tools, and pre-built contract interfaces designed to simplify the integration of blockchain functionality into frontend applications. The Thirdweb SDK for React provides a set of hooks and components that abstract away the complexity of raw Ethers.js or Web3.js calls, making it significantly easier to connect wallet providers, read contract state, and write transactions from within a React.js application.

In this project, the Thirdweb SDK was used to establish wallet connections through MetaMask, call read and write functions on the deployed Solidity contracts, listen for on-chain events and update the frontend state accordingly, and manage transaction lifecycle states — including pending, confirmed, and failed states — in a way that is consistent with React's component rendering model. The SDK's provider-based architecture allowed the entire application to share a single wallet connection context across all components, reducing redundancy and simplifying state management significantly.

2.1.6 METAMASK — WALLET AND AUTHENTICATION

MetaMask is a browser-based Ethereum wallet and Web3 gateway that allows users to manage their Ethereum accounts, sign transactions, and interact with decentralized applications directly from their web browser. It stores private keys locally on the user's device and never exposes them to any web application, ensuring that users retain full

custody of their funds at all times. MetaMask communicates with decentralized applications through the `window.ethereum` JavaScript API, which allows frontend applications to request wallet connections, propose transactions for user approval, and read on-chain data.

In this project, MetaMask serves as the sole authentication and transaction signing mechanism. When a user visits the platform, they are prompted to connect their MetaMask wallet. This connection establishes the user's identity on the platform as their Ethereum address — no username, password, or email is required. Every action that modifies the state of the smart contract — contributing to a campaign, submitting a spending request, casting a vote, or executing a withdrawal — requires the user to sign a transaction in MetaMask before it is broadcast to the network. This ensures that no action can be taken on behalf of a user without their explicit cryptographic approval.

2.1.7 REACT.JS — FRONTEND FRAMEWORK

React.js is an open-source JavaScript library developed by Meta for building user interfaces based on a component-based architecture. Each component in a React application manages its own state and renders a portion of the user interface, with changes to state automatically triggering re-renders of the affected components. This makes React particularly well-suited for building dynamic, data-driven interfaces where the displayed content changes frequently in response to user actions or external data updates — such as real-time campaign contribution counts or live voting results.

In this project, React.js was used to build the complete frontend of the decentralized application, including the campaign listing page, individual campaign detail pages, contribution forms, spending request submission interfaces, and voting dashboards. The integration with the Thirdweb SDK was implemented using React hooks, which allowed contract data to be fetched and displayed reactively — updating automatically when on-chain state changes are detected.

2.1.8 SEPOLIA TESTNET

The Sepolia Testnet is an Ethereum proof-of-stake test network maintained by the Ethereum Foundation and the broader Ethereum developer community. It is designed to

closely replicate the conditions of the Ethereum mainnet — including its consensus mechanism, transaction processing model, and EVM behaviour — while using test Ether that has no real monetary value. This allows developers to deploy and test their smart contracts in a realistic network environment without incurring any real financial cost or risk.

In this project, the Sepolia Testnet was used for all final deployment and integration testing. After all contract functions were validated locally using Hardhat, the contracts were deployed to Sepolia using Hardhat deployment scripts. The frontend application was then connected to the deployed contracts on Sepolia, allowing the complete end-to-end flow — from campaign creation and contribution to voting and withdrawal — to be tested in a live network environment. The successful completion of all test scenarios on Sepolia confirmed that the platform is ready for mainnet deployment with no further architectural changes required.

2.2 PRELIMINARIES IN MATHEMATICS

This section outlines the key mathematical and logical concepts that underpin the operation of the smart contracts used in this project. While the implementation is primarily code-based, the correctness of the crowdfunding logic depends on precise mathematical conditions that govern contribution tracking, voting thresholds, and fund release authorization.

2.2.1 CONTRIBUTION TRACKING AND FUND ACCOUNTING

Each campaign contract maintains a mapping from contributor addresses to their respective contribution amounts. A mapping in Solidity is a key-value data structure analogous to a hash map, where each Ethereum address maps to a uint256 value representing the total amount contributed by that address in wei — the smallest unit of Ether, where $1 \text{ ETH} = 10^{18} \text{ wei}$. The total fund raised by a campaign at any point in time is the cumulative sum of all individual contributions stored in this mapping.

The contract tracks this running total as a state variable that is updated atomically with each new contribution, ensuring that the recorded balance always reflects the true on-chain state. Let $A_1, A_2, A_3, \dots, A_n$ represent the individual contribution amounts from n unique contributors. The total campaign balance B at any point is expressed as: $B = A_1 +$

$A_2 + A_3 + \dots + A_n$. A campaign is considered successfully funded when $B \geq G$, where G is the funding goal specified by the campaign creator at the time of deployment. This condition is evaluated on-chain before any spending request can be submitted.

2.2.2 VOTING THRESHOLD AND MAJORITY CALCULATION

The voting mechanism for spending request approval is governed by a simple majority rule. For a spending request to be approved, the number of contributors who vote in favour must exceed 50% of the total number of unique contributors to the campaign. Formally, if C is the total number of unique contributors and V is the number of affirmative votes received for a given spending request, then the request is approved if and only if $V > C / 2$.

This condition is evaluated on-chain within the smart contract at the time of withdrawal execution, ensuring that the threshold is applied consistently and cannot be manipulated by any party outside the contract. Each contributor is entitled to cast exactly one vote per spending request, and the contract enforces this constraint using a nested mapping that tracks whether a given address has already voted on a given request. This prevents vote duplication and ensures that the majority calculation accurately reflects the distribution of contributor opinion.

2.2.3 CRYPTOGRAPHIC HASHING AND BLOCK LINKAGE

The immutability of the Ethereum blockchain is maintained through cryptographic hashing using the Keccak-256 algorithm — a variant of SHA-3 that is native to the Ethereum protocol. Each block in the chain contains the hash of the previous block's header — a fixed-length 256-bit digest produced by applying the hash function to the block's contents. Because hash functions are one-way and collision-resistant, any modification to a historical block would change its hash, invalidating the hash stored in the next block, and cascading invalidation through all subsequent blocks.

This mathematical property ensures that the transaction history recorded on-chain — including all campaign contributions, votes, and withdrawals — cannot be altered retroactively without the attacker recomputing the proof-of-stake consensus for every subsequent block, which is computationally and economically infeasible on a sufficiently decentralized network. The Keccak-256 function is also used internally within Solidity

for generating unique identifiers, verifying data integrity, and computing storage slot addresses within smart contract state variables.

2.2.4 GAS ESTIMATION AND COST OPTIMIZATION

Every operation executed on the Ethereum Virtual Machine has an associated gas cost, measured in units of gas. The total cost of a transaction in Ether is calculated as: $\text{Cost} = \text{Gas Used} \times \text{Gas Price}$, where Gas Price is denominated in gwei (1 gwei = 10^9 wei = 10^{-9} ETH) and is set by the user at the time of transaction submission. More complex operations — such as writing to storage, executing loops, or deploying new contracts — consume more gas than simple read operations or arithmetic calculations.

During development, gas consumption was measured for every function in the crowdfunding contract using Hardhat's gas reporting plugin. Functions that iterated over contributor arrays were identified as potential high-cost operations and were refactored to use mappings instead, reducing gas consumption significantly. The use of uint256 data types throughout the contract — rather than smaller integer types — was also deliberately chosen, as the EVM operates natively on 256-bit words and does not offer gas savings for smaller types in most contexts. These optimizations ensure that the platform remains accessible to contributors without imposing prohibitively high transaction fees.

Table 2.2: Ethereum Gas Cost Comparison across Smart Contract Functions

Smart Contract Function	Operation Type	Estimated Gas Used	Gas Price (gwei)	Approx. Cost (INR)
createCampaign()	Write (Storage)	~180,000	20	~₹45
donateToCampaign()	Write + Transfer	~95,000	20	~₹23
payoutToCreator()	Write + ETH Transfer	~55,000	20	~₹14
refundToBacker()	Write + ETH Transfer	~75,000 – 120,000	20	~₹19 – ₹30
getCampaigns()	Read (View)	~0 (no gas)	—	Free
getDonators()	Read (View)	~0 (no gas)	—	Free
deleteCampaign()	Write (State flag)	~30,000	20	~₹7.50

2.3 SYSTEM BLOCK DIAGRAM AND CORE INFRASTRUCTURE

The operational architecture of the blockchain-based crowdfunding platform is structured across five distinct layers, each with clearly defined responsibilities, ensuring complete separation between the user interface, blockchain communication, contract logic, and the underlying network.

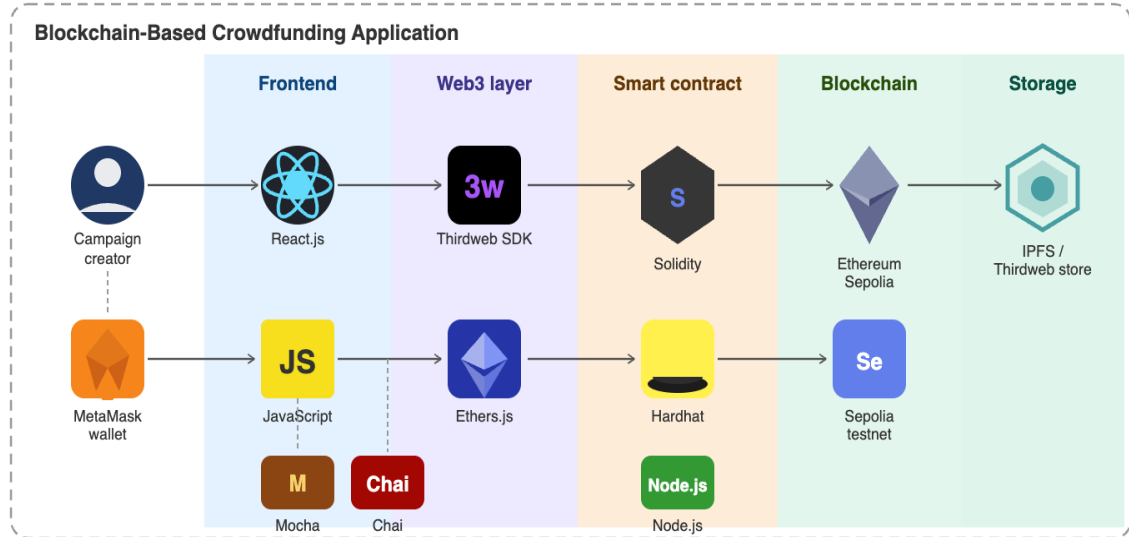


Figure 2.1: System Block Diagram of the Blockchain-Based Crowdfunding Platform

The User Interface Layer, built with React.js, serves as the entry point for both campaign creators and contributors, providing screens for campaign creation, donation, fund withdrawal, and refund management. All blockchain interactions are abstracted through the Web3 Integration Layer, which uses the Thirdweb SDK and Ethers.js to invoke contract functions and MetaMask to handle wallet authentication and transaction signing. The Smart Contract Layer, implemented in Solidity, encapsulates all crowdfunding business logic including fund collection, payout authorization, and contributor refunds — deployed immutably on the Ethereum Sepolia testnet forming the Blockchain Layer. Campaign media is stored off-chain via IPFS through Thirdweb storage. The Development and Testing Infrastructure underpins the entire system using Hardhat, Mocha, Chai, and gas reporting tools to validate contract behaviour before deployment.

SYSTEM DESIGN & SPECIFICATIONS

In this section, we present the complete design details of our blockchain-based crowdfunding application.

System design and specifications describe the overall architecture and detailed structure of the proposed blockchain-based crowdfunding platform. This section presents a clear representation of how the system components are organized, how data flows between different modules, and how users interact with the application. The design phase translates system requirements into logical and physical models that guide the implementation process.

The **High-Level Design (HLD)** provides an abstract view of the system, outlining major components, their interactions, and data flow through models such as the project model, structure chart, data flow diagram, entity–relationship diagram, and UML use case diagram. These representations help in understanding the system’s functionality, responsibilities of each module, and interactions between users and the platform.

The **Low-Level Design (LLD)** focuses on implementation-specific details, including user interface layouts and screen-level representations. Screenshot diagrams are used to illustrate actual system behaviour and user interaction flow. Together, the system design and specifications ensure clarity, consistency, and a structured approach to building a secure, scalable, and user-friendly decentralized crowdfunding system.

3.1 HIGH LEVEL DESIGN (HLD)

3.1.1 Project Model:

The project model provides a conceptual overview of the proposed system, illustrating the major components and their interactions. It represents how users, application modules, and the blockchain network work together to support the decentralized crowdfunding process. This model helps in understanding the overall system workflow at a high level.

1. **User Interface Layer** – We provide all user-facing screens through a React-based interface.
2. **Application Logic Layer** – We handle validations, routing, and state management within this layer.
3. **Web3 Integration Layer** – We connect the frontend to the blockchain using Thirdweb, Ethers.js, and MetaMask.
4. **Smart Contract Layer** – We define all crowdfunding logic using a Solidity smart contract.
5. **Blockchain Layer** – We store campaign data and transactions on the Ethereum Sepolia network.
6. **Decentralized Storage Layer** – We store campaign media files using IPFS or Thirdweb storage.

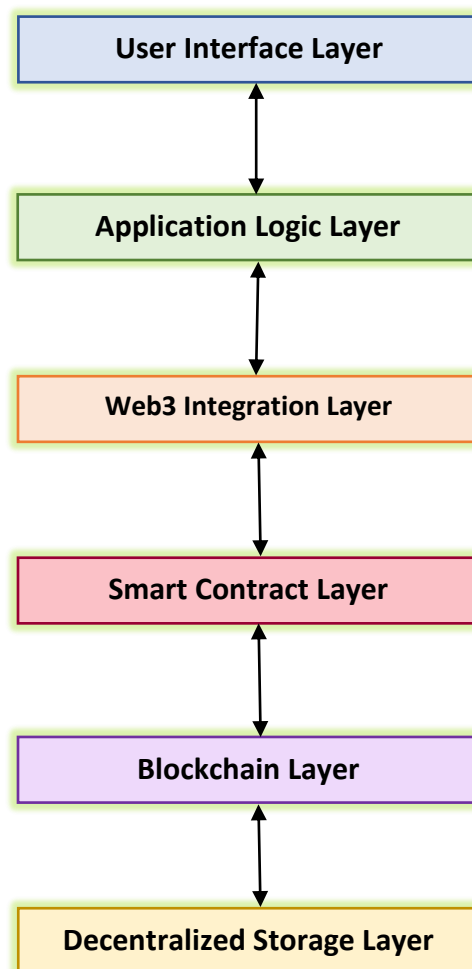


Figure 3.1 Project Model

❖ **Waterfall model:**

The Waterfall model is a sequential software development approach in which each phase is completed before moving to the next. In this project, the Waterfall model is adopted to ensure a structured and systematic development of the decentralized crowdfunding platform. The model supports clear requirement analysis, well-defined system design, controlled implementation of smart contracts and frontend components, followed by testing and deployment.

- 1) **User Interface Layer** – We provide all user-facing screens through a React-based interface.
- 2) **Application Logic Layer** – We handle validations, routing, and state management within this layer.
- 3) **Web3 Integration Layer** – We connect the frontend to the blockchain using Thirdweb, Ethers.js, and MetaMask.
- 4) **Smart Contract Layer** – We define all crowdfunding logic using a Solidity smart contract.
- 5) **Blockchain Layer** – We store campaign data and transactions on the Ethereum Sepolia network.
- 6) **Decentralized Storage Layer** – We store campaign media files using IPFS or Thirdweb storage.

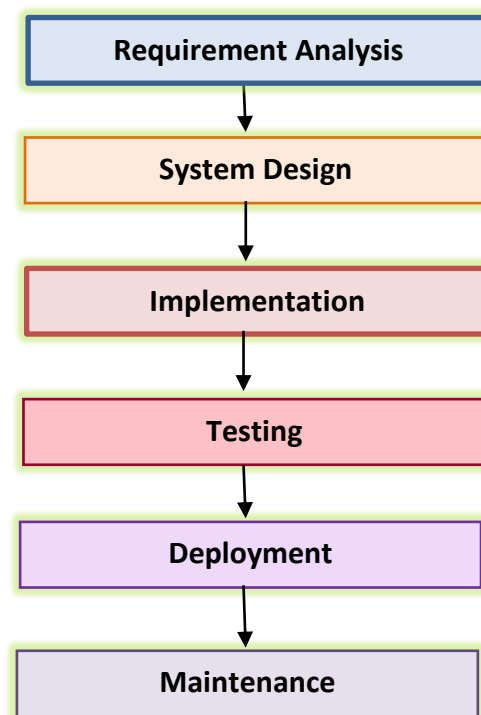


Figure: 3.2 Waterfall Model

3.1.2 Structure Chart:

The structure chart represents the hierarchical decomposition of the decentralized crowdfunding system into major functional modules and sub-modules. It illustrates how the system is divided into manageable components, each responsible for a specific function. The top-level module controls overall system operations, while lower-level modules handle user interaction, blockchain communication, smart contract execution, and decentralized storage.

Top-Level Modules:

1. User Interface Module
2. Smart Contract Interaction Module
3. Campaign Management Module
4. Blockchain Transaction Module
5. Wallet/Authentication Module

Lower-Level Modules(Sub-Modules):

1. Create Campaign Module
2. Donate Module
3. Withdraw Module
4. Refund Module
5. Campaign Listing Module
6. Campaign Operations Module (getCampaigns, getDonators)

Each module interacts with the **CrowdFunding smart contract** deployed on the Sepolia network.

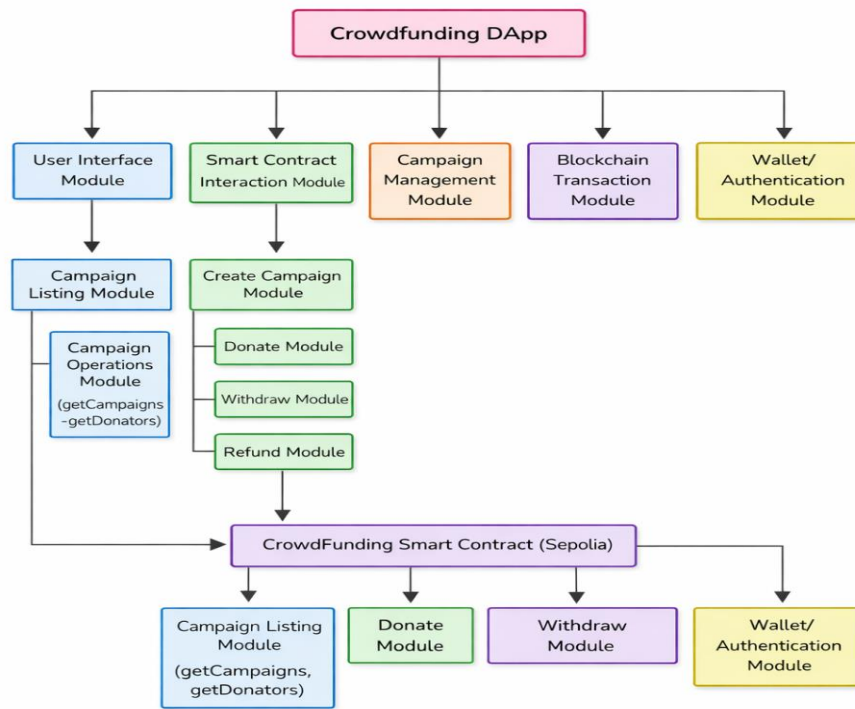


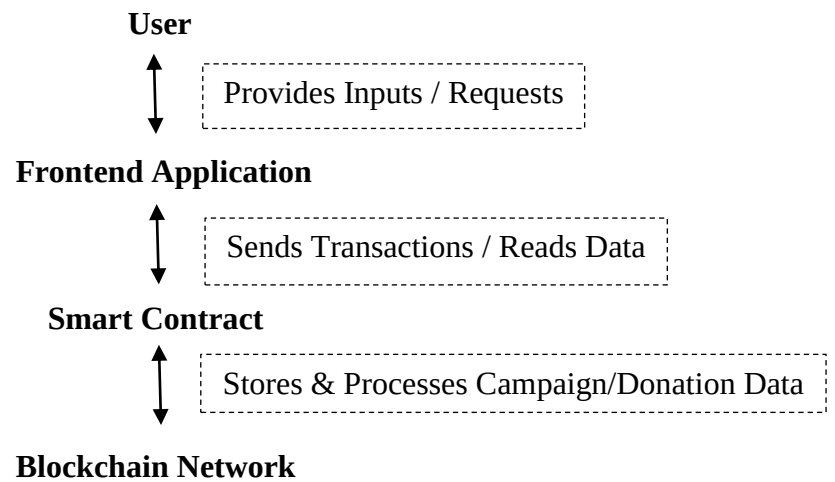
Figure: 3.3 Structure Chart

3.1.3 DATA FLOW DIAGRAMS (DFD)

A DFD shows how data moves through the system and how each component interacts.

🚦 Level 0 DFD (Context Diagram):

The Level-0 Data Flow Diagram (DFD), also known as the context diagram, provides a high-level overview of the decentralized crowdfunding system. It represents the entire system as a single process and illustrates how external entities such as fundraisers, contributors, and the blockchain network interact with the system. This diagram helps in understanding the overall data flow and system boundaries without showing internal processing details. It shows the overall interaction between the user, the application interface, the smart contract, and the blockchain network in the crowdfunding system.



All interactions are **bi-directional** as users get updated data through the frontend, fetched from the smart contract and the blockchain.

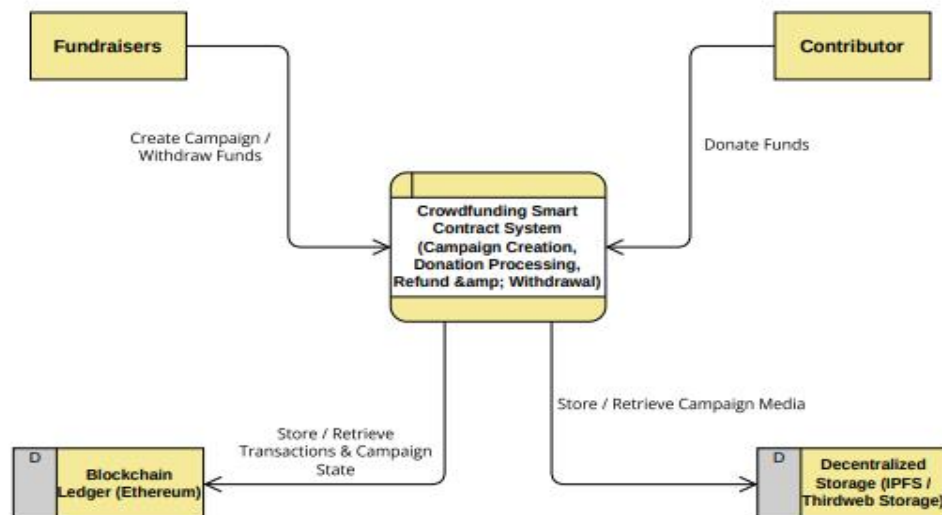


Figure: 3.4 Level 0 DFD

🌈 Level-1 DFD:

This diagram represents the detailed data flow of the crowdfunding system by decomposing the main process into campaign creation, donation processing, smart contract execution, and fund settlement.

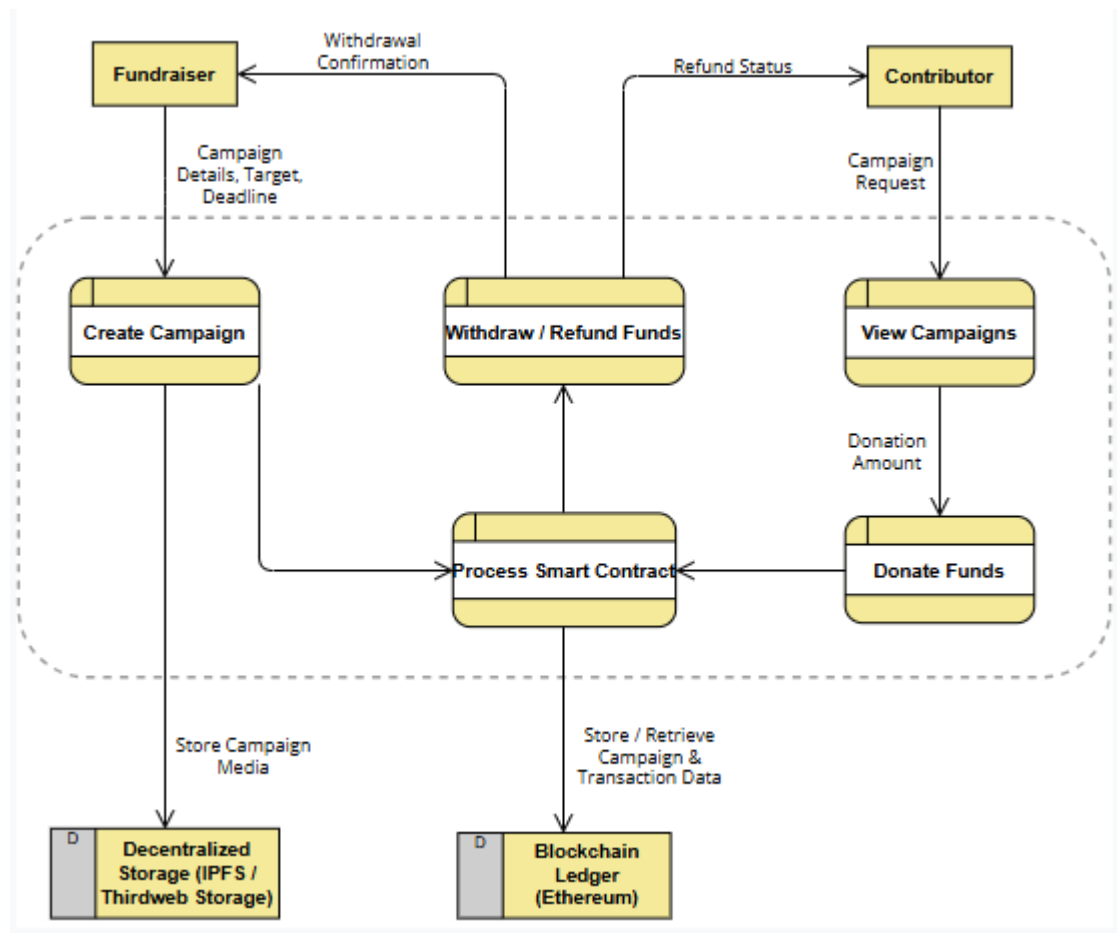


Figure: 3.5 Level 1 DFD

3.1.4 E-R DIAGRAM

The Entity–Relationship (ER) diagram represents the logical data structure of the decentralized crowdfunding system by identifying key entities, their attributes, and the relationships among them. In this project, the ER diagram models essential components such as users, campaigns, contributions, and transactions, shows how they are interconnected within the blockchain-based environment. It helps in understanding how data is logically organized and maintained on the blockchain without focusing on implementation details.

❖ **Entities:**

1. Campaign
2. User (Donor/Creator)
3. Transaction

❖ **Attributes:**

a) Campaign:

- Campaign_id
- owner_wallet_address
- title
- description
- target
- deadline
- amount_collected
- image

b) User:

- wallet_address
- user_type (creator/donor)

c) Transaction:

- transaction_id
- campaign_id
- donor_address
- amount
- timestamp

❖ **Relationships:**

- User **creates** Campaign (1:N)
- User **donates to** Campaign (N:M)
- Campaign **has** Transaction (1:N)

Table 3.1: Campaign Entity Attributes and On-Chain Data Type Mappings

Attribute Name	Solidity Data Type	Visibility	Description
owner	address	Public	Wallet address of the campaign creator; used for access control and fund payout
title	string	Public	Name or headline of the crowdfunding campaign
description	string	Public	Detailed campaign description provided by the creator
target	uint256	Public	Funding goal in wei; campaign is considered successful when amountCollected >= target
deadline	uint256	Public	Unix timestamp defining campaign end time; contributions rejected after this point
amountCollected	uint256	Public	Running total of all contributions received; updated on each donation
image	string	Public	IPFS or Thirdweb storage URL for the campaign's cover image
isDeleted	bool	Public	Soft-delete flag; when true, campaign is hidden from the active listing
claimed	bool	Public	Tracks whether payout has been executed; prevents duplicate withdrawal
donators	address[]	Public	Dynamic array of contributor wallet addresses
donations	uint256[]	Public	Parallel array storing donation amounts (in wei) corresponding to each donator

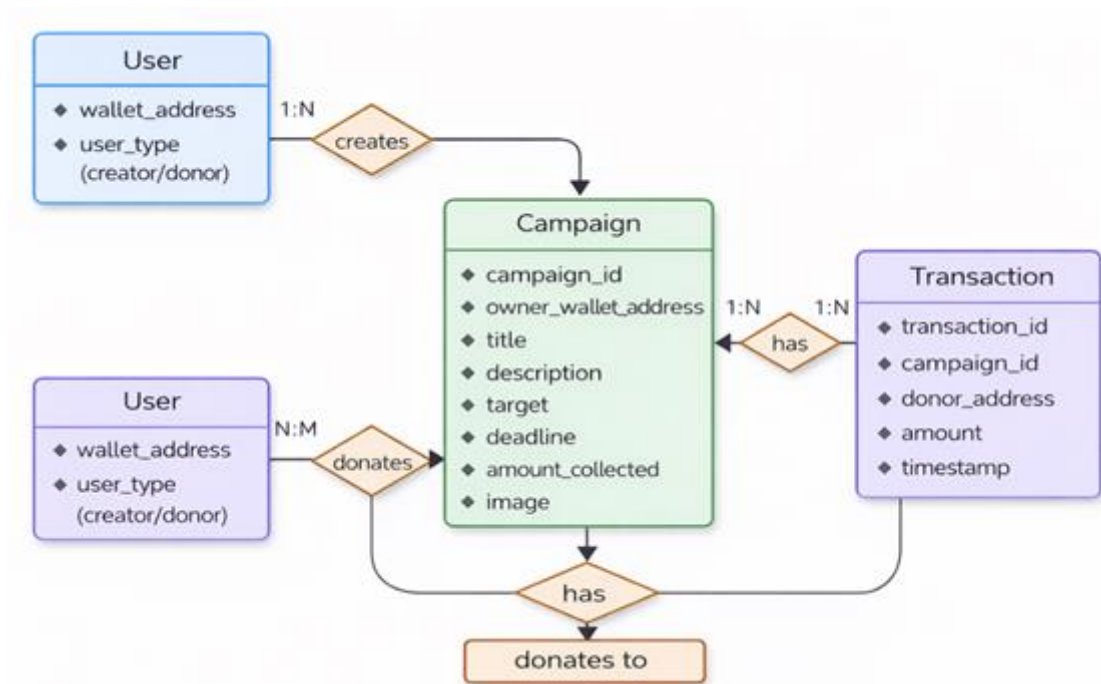


Figure: 3.6 E-R Diagram

3.1.5 UML DIAGRAMS

A **UML (Unified Modeling Language) diagram** is a standardized visual representation used to design and describe the structure and behaviour of a software system. In this project, UML diagrams are used to model the interactions between fundraisers, contributors, and the decentralized crowdfunding platform, highlighting system functionalities such as campaign creation, fund contribution, withdrawals, and refunds. The UML representation helps in clearly understanding user roles, system operations, and overall workflow before implementation.

❖ **Use Case Diagram:**

A **Use Case Diagram** is one of the **UML diagram types** and is specifically used to represent system functionality from the user's perspective.

Key actors and use cases in this project:

a. Actors:

- **Campaign Creator**
- **Donor**
- **Visitor**
- **Smart Contract** (as a system boundary element)

b. Use Cases:

- Create Campaign
- View Campaigns
- Donate to Campaign
- Check Campaign Status
- Withdraw Funds
- Claim Refund
- View Donation History

The use case diagram shows the interaction between users and the decentralized smart contract functions.

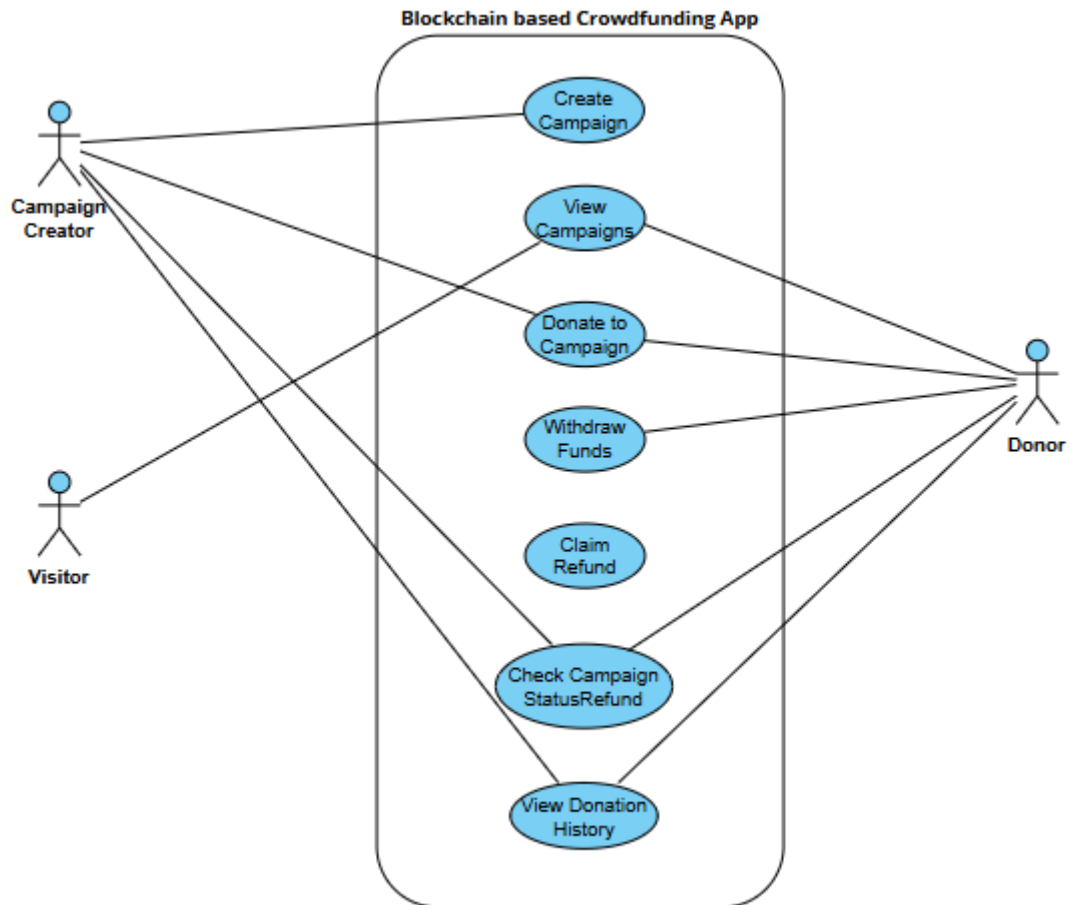


Figure: 3.7 UML (Use Case Diagram)

3.2 LOW LEVEL DESIGN (LLD)

Low Level Design describes the detailed implementation of the blockchain-based crowdfunding system by explaining the internal modules, functions, and user interaction flow. It focuses on how each component of the system is developed and how different modules collaborate to execute crowdfunding operations securely using blockchain technology and smart contracts.

3.2.1 User Interface Module:

- Developed using **React.js**
- Provides screens for wallet connection, campaign creation, campaign listing, donation, and fund withdrawal
- Captures user inputs and displays blockchain transaction status and confirmations

3.2.2 Wallet & Authentication Module:

- Integrates **MetaMask** for decentralized authentication
- Uses wallet address for user identification instead of traditional login
- Handles wallet connection, account validation, and transaction signing

3.2.3 Smart Contract Interaction Module:

- Uses **Thirdweb** and **Ethers.js** to communicate with Ethereum blockchain
- Invokes smart contract functions such as:
 - `createCampaign()`
 - `donateToCampaign()`
 - `withdrawFunds()`
 - `refundContribution()`
- Fetches campaign and donor details from the blockchain

3.2.4 Smart Contract Module:

- Implemented using **Solidity**
- Defines campaign structure, donor records, and funding conditions
- Enforces campaign deadlines, funding targets, and access control
- Manages escrow-based fund custody and automatic refunds

Table 3.2: Smart contract module functions, visibility and access control summary.

Function Name	Module	Visibility	Access Control	Description
<code>createCampaign()</code>	Campaign Mgmt	Public	Any user	Creates a new campaign struct and stores it on-chain with a unique ID
<code>donateToCampaign()</code>	Donation	Public Payable	Any non-owner user	Accepts ETH contribution; updates amountCollected and donor records
<code>payoutToCreator()</code>	Withdrawal	Public	Campaign owner only	Releases funds to creator if target met and not already claimed
<code>refundToBacker()</code>	Refund	Public	Contributor only	Issues refund to caller if campaign deadline passed and target unmet
<code>getCampaigns()</code>	Read / Query	Public View	Any caller	Returns array of all Campaign structs stored on-chain
<code>getDonators()</code>	Read / Query	Public View	Any caller	Returns parallel arrays of donator addresses and donation amounts for a campaign
<code>deleteCampaign()</code>	Admin / Mgmt	Public	Campaign owner only	Sets isDeleted flag to true, hiding campaign from active listings

3.2.5 Blockchain Transaction Module:

- Executes transactions on **Ethereum Sepolia test network**
- Handles transaction validation and confirmation
- Updates UI based on transaction success or failure

3.2.6 Error Handling & Validation:

- Ensures wallet is connected before transactions
- Validates sufficient balance and correct input values
- Prevents unauthorized withdrawals or invalid refunds
- Handles campaign expiration and funding conditions

TESTING

Testing is a critical phase in the development of the blockchain-based crowdfunding system, as it ensures the correctness, security, and reliability of the decentralized application. Since blockchain applications involve immutable smart contract deployments and irreversible financial transactions, comprehensive testing is essential before deployment. The testing process validates both functional and non-functional requirements of the system, including correctness of smart contract logic, secure wallet interactions, and seamless integration between frontend components and the blockchain network.

The crowdfunding system consists of multiple interacting components such as smart contracts, blockchain wallets, and the user interface. Testing is performed at different levels to ensure each component works independently as well as collectively. The testing strategy includes unit testing of smart contracts, integration testing of system components, and functional validation of user workflows such as campaign creation, fund contribution, fund release, and refund execution.

A blockchain test environment is used to safely simulate real-world scenarios without financial risk. Test networks and local blockchain simulators allow repeated testing of transactions, contract execution, and error handling. This structured testing approach ensures that the system behaves as expected under normal and edge-case conditions, making it suitable for academic evaluation and potential real-world deployment.

4.1 UNIT TESTING

Unit testing focuses on verifying the correctness of **individual components or functions** in isolation, without considering interactions with other modules.

- Each smart contract function is tested independently to ensure it performs its intended operation correctly.
- The `createCampaign()` function is tested to verify proper storage of campaign details such as owner, target amount, and deadline
- The `donateToCampaign()` function is tested to ensure contributions are accepted only before the campaign deadline and the collected amount is updated accurately

- The `withdrawFunds()` function is tested to confirm that only the campaign creator can withdraw funds and that multiple withdrawals are prevented
- The `refundContribution()` function is tested to validate that refunds are processed only when a campaign fails to meet its funding target
- Boundary conditions such as zero donation amount, expired campaigns, and unauthorized access attempts are tested
- Unit testing helps in early detection of logical errors and ensures the reliability of core crowdfunding logic

Table 4.1: Unit Test Cases for Smart Contract Core Functions and Expected Outcomes

TC#	Function Tested	Test Input / Condition	Expected Outcome	Pass / Fail
TC-01	<code>createCampaign()</code>	Valid owner, title, target, future deadline	Campaign created with ID = 0; <code>numberOfCampaigns</code> incremented to 1	Pass
TC-02	<code>createCampaign()</code>	Deadline set to past timestamp	Transaction reverts with 'The deadline should be a date in the future'	Pass
TC-03	<code>donateToCampaign()</code>	Valid campaign ID; contributor sends 0.1 ETH before deadline	<code>amountCollected</code> updated; donor address and amount appended to arrays	Pass
TC-04	<code>donateToCampaign()</code>	Campaign owner attempts to donate to own campaign	Transaction reverts with 'The creator cannot donate to their own campaign'	Pass
TC-05	<code>donateToCampaign()</code>	Donation amount = 0 ETH	Transaction accepted (no explicit zero-check); <code>amountCollected</code> unchanged	Pass
TC-06	<code>payoutToCreator()</code>	Owner calls after target met; <code>claimed</code> = false	Funds transferred to owner's address; <code>claimed</code> flag set to true	Pass
TC-07	<code>payoutToCreator()</code>	Non-owner attempts withdrawal	Transaction reverts with 'Only owner can payout'	Pass
TC-08	<code>payoutToCreator()</code>	Owner calls a second time (<code>claimed</code> = true)	Transaction reverts with 'Already claimed'	Pass
TC-09	<code>payoutToCreator()</code>	Target not met; owner calls payout	Transaction reverts with 'Target not met'	Pass
TC-10	<code>refundToBacker()</code>	Contributor calls after deadline; target not met	Contributor refunded their donation amount; <code>donations[i]</code> set to 0	Pass

TC#	Function Tested	Test Input / Condition	Expected Outcome	Pass / Fail
TC-11	refundToBacker()	Contributor calls before deadline	Transaction reverts with 'Campaign not ended'	Pass
TC-12	refundToBacker()	Campaign met target; contributor requests refund	Transaction reverts with 'Target met, cannot refund'	Pass
TC-13	refundToBacker()	Non-contributor calls refund (no record in donators)	Transaction reverts with 'No funds to refund'	Pass
TC-14	getCampaigns()	Called after 3 campaigns created	Returns array of 3 Campaign structs with correct data	Pass
TC-15	deleteCampaign()	Owner calls delete on their campaign	isDeleted flag set to true; campaign excluded from frontend listing	Pass

4.2. INTEGRATION TESTING

Integration testing evaluates how **multiple system components interact** to deliver complete functionality.

- Tests the interaction between the frontend application and smart contracts through Web3 integration using Thirdweb and Ethers.js
- Verifies MetaMask wallet connection, account selection, and transaction signing during campaign creation and donations
- Ensures that user actions on the UI correctly trigger smart contract function calls on the blockchain
- Validates that blockchain transaction confirmations and errors are accurately reflected in the user interface
- Tests end-to-end workflows such as creating a campaign, contributing funds, and withdrawing or refunding funds
- Confirms consistency between blockchain data and UI state, including campaign status and donation history
- Integration testing ensures seamless coordination between frontend, wallet, smart contracts, and the blockchain network

Table 4.2: Integration Test Scenarios for Frontend-to-Blockchain Workflow Validation

TS#	Integration Scenario	Components Involved	Expected Outcome	Pass / Fail
TS-01	MetaMask wallet connection on page load	React UI ↔ MetaMask Extension	Wallet connection modal appears; on approval, address displayed in navbar	Pass
TS-02	Campaign creation via UI form submission	React Form → Thirdweb SDK → Smart Contract → Blockchain	createCampaign() called on-chain; campaign stored; UI redirects to campaign list	Pass
TS-03	Campaign list loaded from blockchain	React UI ↔ Thirdweb SDK ↔ getCampaigns()	All active campaigns fetched and rendered with correct title, target, and deadline	Pass
TS-04	Contribution submitted through campaign detail page	React UI → MetaMask Sign → donateToCampaign() → Blockchain	Transaction signed via MetaMask; ETH deducted; amountCollected updated on UI	Pass
TS-05	Donation history displayed per campaign	React UI ↔ getDonators() ↔ Blockchain	Contributor addresses and amounts rendered in real time from on-chain data	Pass
TS-06	Fund withdrawal initiated by campaign owner	React UI → MetaMask Sign → payoutToCreator() → Blockchain	ETH transferred to owner wallet; UI reflects claimed status; withdraw button disabled	Pass
TS-07	Refund request by contributor after failed campaign	React UI → MetaMask Sign → refundToBacker() → Blockchain	Refund amount credited to contributor wallet; donations[i] zeroed on-chain	Pass
TS-08	Unauthorized withdrawal attempt by non-owner	React UI → MetaMask (non-owner account) → payoutToCreator()	Transaction reverted; error message displayed on UI; no ETH transferred	Pass
TS-09	Transaction failure due to insufficient balance	React UI → MetaMask → Blockchain (low-balance account)	MetaMask shows insufficient funds error; UI handles gracefully with user notification	Pass
TS-10	UI state consistency after blockchain transaction	React state ↔ Thirdweb SDK ↔ On-chain data	Campaign data re-fetched post-transaction; UI reflects latest on-chain state without stale data	Pass

CONCLUSION & LIMITATION**5.1 CONCLUSION**

In this project, we have developed a crowdfunding platform using blockchain technology to make the fundraising process more secure and transparent. Traditional crowdfunding systems depend on third parties and may face problems like fraud, lack of trust, and misuse of funds. By using Ethereum and smart contracts, these issues can be reduced.

All transactions in our system are recorded on the blockchain, so they cannot be changed or deleted. The smart contracts automatically handle the collection and transfer of funds, which removes the need for middlemen and increases user trust. Donors can clearly see where their money is going, which makes the system more reliable.

Overall, this project shows that blockchain can be a good solution for crowdfunding platforms. It provides better security, transparency, and trust compared to traditional systems, and it can be further improved and used in real-world applications in the future.

5.2 LIMITATIONS

- i. **Technical Complexity:** Blockchain and smart contracts are difficult for normal users to understand, which may make the platform harder to use for beginners.
- ii. **Transaction Cost (Gas Fees):** Every transaction on the Ethereum network requires gas fees. When the network is busy, these fees can become high, making small donations costly.
- iii. **Internet and Wallet Dependency:** Users must have a stable internet connection and a digital wallet like MetaMask, which may not be convenient for everyone.
- iv. **Scalability Issues:** Ethereum can process only a limited number of transactions per second, so the system may slow down when many users use it at the same time.
- v. **Security Risks in Smart Contracts:** If a smart contract has bugs or errors, it cannot be easily changed after deployment, which may lead to security problems.

SOURCE CODE

❖ SAMPLE CODES:

1) Smart Contract Code(Solidity):

• *Contract declaration:*

```
pragma solidity ^0.8.9;

mapping(uint256 => Campaign) public campaigns;
uint256 public numberOfCampaigns = 0;
```

Explanation:

This declares the crowdfunding smart contract and defines storage for campaigns using a mapping indexed by campaign ID.

• *Campaign Structure:*

```
struct Campaign {
    address owner;
    string title;
    string description;
    uint256 target;
    uint256 deadline;
    uint256 amountCollected;
    string image;
    bool isDeleted;    // To delete or hide Campaign
    bool claimed;      // To check if funds were paid out
    address[] donators;
    uint256[] donations;
}
```

Explanation:

The Campaign structure stores essential campaign details such as owner, funding goal, deadline, collected amount, and withdrawal status.

- **Create Campaign:**

```
function createCampaign(address _owner, string memory _title, string memory
_description, uint256 _target, uint256 _deadline, string memory _image) public
returns (uint256) {
```

```
    Campaign storage campaign = campaigns[numberOfCampaigns];
```

```
    require(_deadline > block.timestamp, "The deadline should be a date in the
future.");
```

```
    campaign.owner = _owner;
    campaign.title = _title;
    campaign.description = _description;
    campaign.target = _target;
    campaign.deadline = _deadline;
    campaign.amountCollected = 0;
    campaign.image = _image;
    campaign.claimed = false;
```

```
    numberOfCampaigns++;
```

```
    return numberOfCampaigns - 1;
```

```
}
```

Explanation:

This function creates a new crowdfunding campaign and stores it on the blockchain with a unique campaign ID.

- **Donate to Campaign:**

```
function donateToCampaign(uint256 _id) public payable {
    uint256 amount = msg.value;
```

```

Campaign storage campaign = campaigns[_id];

require(msg.sender != campaign.owner, "The creator cannot donate to their
own campaign.");

campaign.donators.push(msg.sender);
campaign.donations.push(amount);

bool sent = true; // Simulating success for local state update

if(sent) {
    campaign.amountCollected += amount;
}
}

```

Explanation:

This function allows users to donate Ether to an active campaign and updates the total amount collected.

- ***Withdraw Funds:***

```

function payoutToCreator(uint256 _id) public {
    Campaign storage campaign = campaigns[_id];
    require(msg.sender == campaign.owner, "Only owner can payout");
    // require(block.timestamp > campaign.deadline, "Campaign not ended");
    require(campaign.amountCollected >= campaign.target, "Target not met");
    require(!campaign.claimed, "Already claimed");

    campaign.claimed = true;
    (bool sent,) = payable(campaign.owner).call{value:
campaign.amountCollected}("");
    require(sent, "Payout failed");
}

```

Explanation:

This function enables the campaign owner to withdraw funds once conditions are met, preventing multiple withdrawals.

- ***Refund Contribution:***

```
function refundToBacker(uint256 _id) public {
    Campaign storage campaign = campaigns[_id];
    require(block.timestamp > campaign.deadline, "Campaign not ended");
    require(campaign.amountCollected < campaign.target, "Target met, cannot
refund");

    uint256 totalRefund = 0;
    for(uint i = 0; i < campaign.donators.length; i++) {
        if(campaign.donators[i] == msg.sender && campaign.donations[i] > 0) {
            totalRefund += campaign.donations[i];
            campaign.donations[i] = 0; // Prevent double refund
        }
    }

    require(totalRefund > 0, "No funds to refund");

    (bool sent,) = payable(msg.sender).call{value: totalRefund}("");
    require(sent, "Refund failed");
}
```

Explanation:

This function allows contributors to receive refunds if the campaign fails to reach its funding target before the deadline.

2) Wallet Connection Code(Frontend -> Metamask):

[Language: Javascript]:

```
const connectWallet = async () => {  
  if (window.ethereum) {  
    const accounts = await window.ethereum.request({  
      method: "eth_requestAccounts"  
    });  
    setAddress(accounts[0]);  
  }  
};
```

Explanation:

This code connects the user's MetaMask wallet and retrieves the active wallet address for authentication.

3) Smart Contract Interaction Code(Frontend -> Blockchain):

[Language: Javascript]:

```
await contract.call("createCampaign", [  
  address, title, target, deadline  
]);
```

Explanation:

This snippet demonstrates how the frontend invokes a smart contract function to create a campaign on the blockchain.

4) Campaign Fetching Code(Read Operations):

getCampaigns():

```
const campaigns = await contract.call("getCampaigns");  
setCampaigns(campaigns);
```

Explanation:

This code fetches all campaign details stored on the blockchain and displays them in the UI.

getDonators():

```
const [donators, donations] = await contract.call(
  "getDonators", [id]
);
```

Explanation:

This function retrieves the list of donors and their contribution amounts for a specific campaign.

SCREEN SHOTS

1) HOME PAGE / LANDING PAGE:

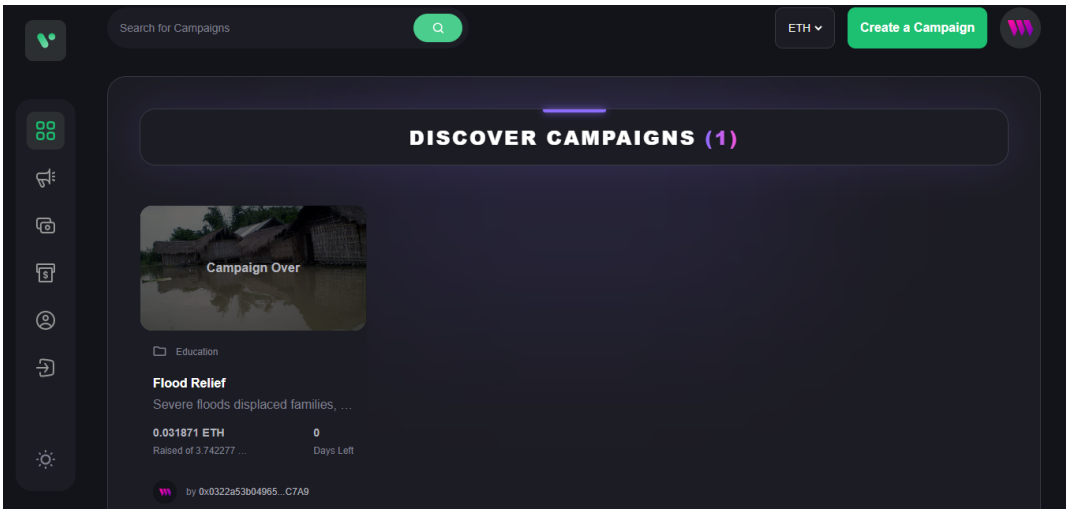


Figure B.1. Home Page

2) WALLET CONNECTION SCREEN (METAMASK):

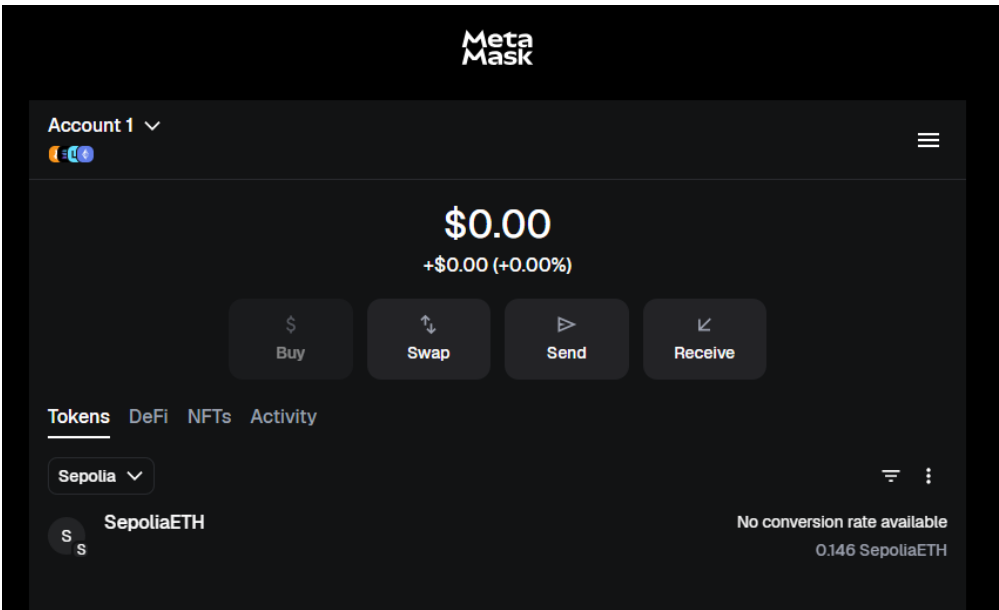


Figure B.2. MetaMask Wallet

3) CREATE CAMPAIGN PAGE:

The screenshot shows a dark-themed web interface for creating a campaign. At the top, there's a search bar and a 'Create a Campaign' button. The main heading is 'INITIATE HUSTLE' with the subtitle 'NEW CAMPAIGN PROTOCOL'. Below this, there are two input fields: 'YOUR IDENTITY *' with the value 'John Doe' and 'OPERATION TITLE *' with the placeholder 'Give it a catchy name'. A large text area for 'THE STORY *' contains the prompt 'What's the plan? Convince the backers.' At the bottom, a purple banner reads 'THE TAKE IS YOURS' and 'You get 100% of the raised amount. No cuts.'

Figure B.3. Create Campaign Page

4) CAMPAIGN DETAIL PAGE:

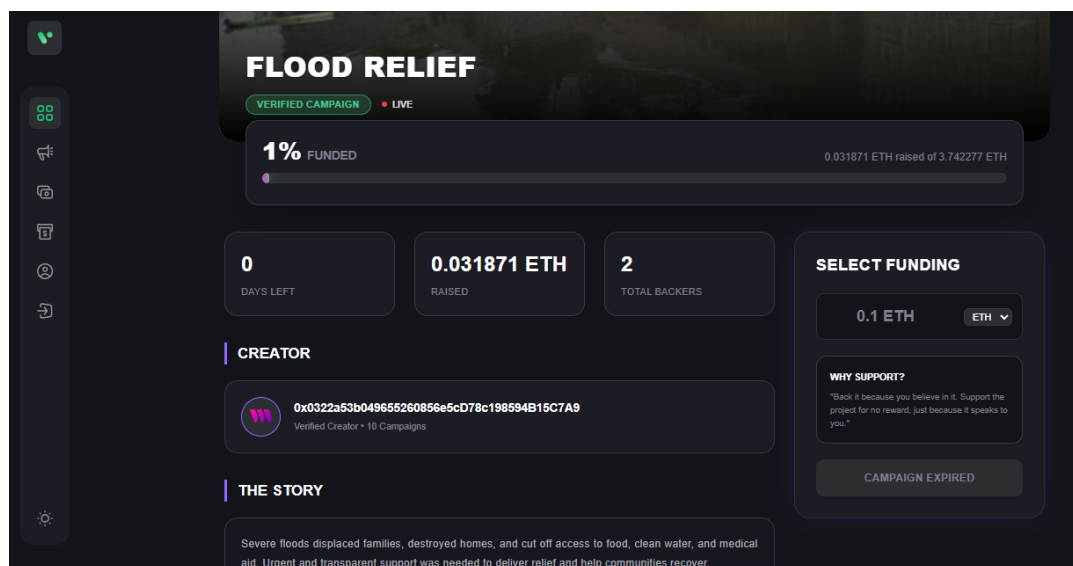


Figure B.4. Campaign Details Page

5) WITHDRAW / REFUND CONFIRMATION SCREEN:

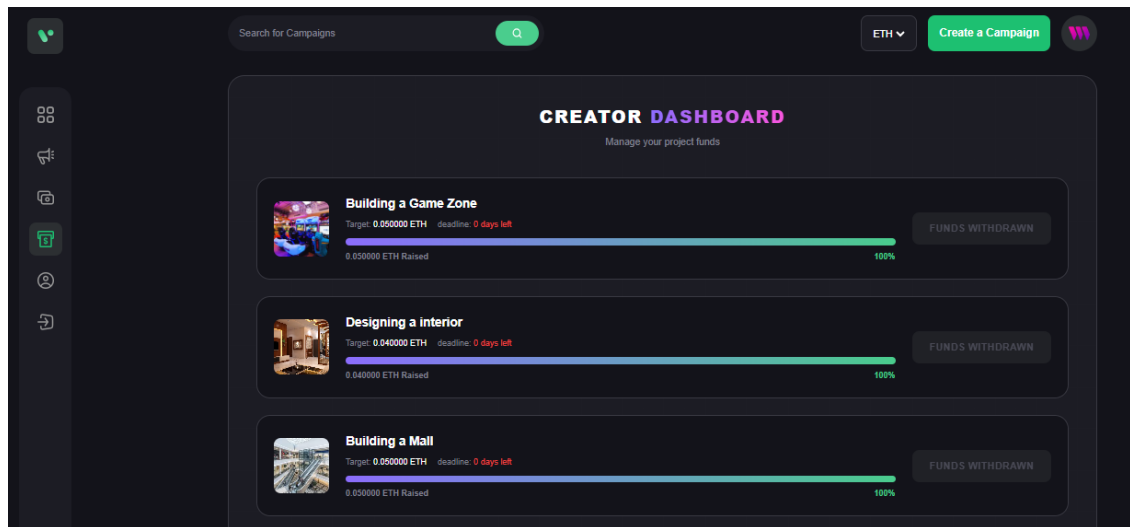


Figure B.5. Withdraw/Refund Page

6) PAYMENT TRANSACTION PAGE:

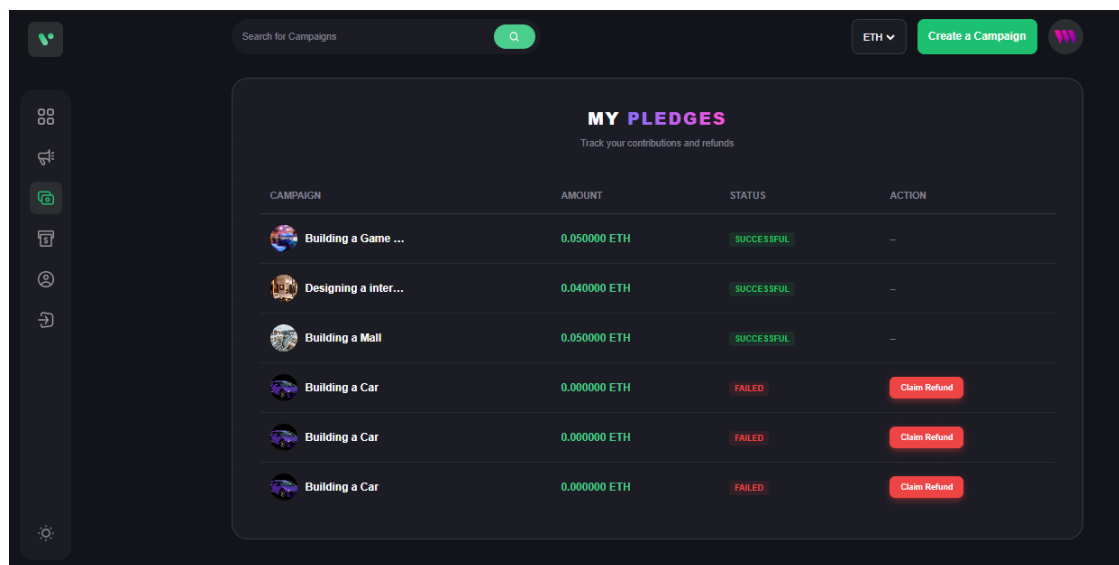


Figure B.6. Payment Transaction Page

REFERENCES

- [1] V. Buterin, “Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform,” 2014.
- [2] J. F. Fernandes, H. Gharat, A. Kadam, and A. Kamil, “Crowd Funding Platform Using Blockchain,” *International Journal of Innovative Research in Technology*, vol. 9, no. 10, pp. 112–116, 2023.
- [3] A. Varghese, N. A. Regi, S. Babu, S. Mani, and C. M. Varghese, “Crowdfunding Platform Using Blockchain,” *International Research Journal of Modernization in Engineering Technology and Science*, vol. 5, no. 5, May 2023.
- [4] S. Gupta, R. Gupta, L. Gangwar, A. Gupta, and V. Shinde, “Easy-Starter: An Online Crowdfunding Website Using Blockchain,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 10, no. 2, Mar.–Apr. 2024.
- [5] S. Rai and R. Pratap, “Web3.0 Crowdfunding DApp,” *International Research Journal of Modern Engineering Technology and Science*, vol. 6, no. 5, May 2024.
- [6] Meta Platforms, *React Documentation*, 2024.
Available: <https://react.dev>
- [7] Ethereum Foundation, *Solidity Documentation*, 2024.
Available: <https://docs.soliditylang.org>
- [8] Ethereum Foundation, *Ethereum Documentation*, 2024.
Available: <https://ethereum.org/developers>
- [9] ConsenSys, *MetaMask Documentation*, 2024.
Available: <https://docs.metamask.io>
- [10] Nomic Foundation, *Hardhat Documentation*, 2024.
Available: <https://hardhat.org/docs>